

Questão 04. Um estudante iniciante de programação em C escreveu o programa abaixo e encontrou diversos erros que impedem a sua compilação. Analise o código atentamente, aponte cada um dos erros presentes e escreva a versão corrigida e funcional desse programa:

em vermelho está o código errado:

```
#include <stdio.h> /*não tem erro aqui*/
#include <stdlib.h>; /*não precisa do “;”*/
int Main{} /*o M é minúsculo e Trocou o “( )” por “{ }” na função main, o correto é int
main(){ }
(
printf( Existem %d semanas no ano.,52); /* Faltaram aspas para separar a saída dos
argumentos*/
cout << endl; /* colocar um \n no fim da saída seria o correto*/
system("PAUSE"); /*não tem erro aqui*/
return 0; /*não tem erro aqui*/
)
```

código corrigido:

```
#include <stdio.h>
#include <stdlib.h>

int main (){

printf(“Existem %d semanas no ano.\n”, 52);

System(“PAUSE”);

return 0;

}
```

Questão 05. Analise o seguinte trecho de código em C. Sob a perspectiva do padrão ANSI C, o programa está correto para compilação e execução imediata? Caso negativo, descreva quais elementos cruciais e diretivas estão faltando no código abaixo:

Em vermelho o código errado:

```
/*Está faltando o cabeçalho <stdio.h> para usar o printf()*/
/*Está faltando o cabeçalho <stdlib.h> para usar o system()*/

main() /*Tá faltando dizer qual é o tipo de retorno da função main (int, void (caso
não tenha retorno) etc.) */
```

```

{
printf("Linguagem C"); /* não tem erro aqui, mas seria ideal colocar um \n no final
para separar da informação que o sistema irá dar depois no system("pause") */
system("pause"); /* não tem erro aqui */
}

```

Código corrigido:

```

#include <stdio.h>
#include <stdlib.h>

```

```

int main() {

printf("Linguagem C");
system("pause");

}

```

Questão 06. Identifique e liste todos os erros de sintaxe (que violam as regras da linguagem C) e de lógica contidos no programa abaixo:

```

/*Está faltando o cabeçalho <stdio.h> para usar o printf()*/
/*Está faltando o csbeçalho <stdlib.h> para usar o system()*/

```

```

main() /*Tá faltando dizer qual é o tipo de retorno da função main (int, void (caso
não tenha retorno) etc.) */
{
int a=1; b=2; c=3:
printf("Os números são: %d%d%d\n, a, b, c, d); /* Faltaram aspas para separar a
saída dos argumentos, esse d não deveria tá ali nos argumentos, tendo em vista
que ele não foi declarado em momento nenhum. faltou também uma virgula entre os
especificadores para separar cada número. Do jeito que está, caso saísse algo,
seria tudo junto "123" e não "1,2,3" */

system("pause"); /* não tem erro aqui */
}

```

Código corrigido:

```

#include <stdio.h>
#include <stdlib.h>

```

```

int main () {

```

```
int a = 1; int b = 2; int c = 3;
printf("Os numeros sao %d,%d,%d\n",a,b,c);
system("pause");
}
```

Questão 07. Descreva a saída exata (incluindo quebras de linha e tabulações) que será impressa no console por cada uma das seguintes instruções independentes do printf():

a) printf("\n\tBom dia! Shirley.");

```
Bom dia! Shirley.
```

b) printf("Você já tomou café? \n");

```
Você já tomou café?
```

c) printf("\n\nA solução não existe!\n\nNão insista.");

```
A solução não existe!
Não insista.
```

d) printf("Duas\tlinhas\tde\tsaída\nou\tuma?");

```
Duas      Linhas      de      saída
ou      uma?
```

e) printf("%s\n%s\n%s\n", "um", "dois", "três");

```
um
dois
três
```

Questão 08. Explique detalhadamente o comportamento do programa abaixo quando executado no console. Apresente qual será a saída exata gerada pelas sequências de escape utilizadas no formato de controle:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
printf("\n\t\"Primeiro programa\");
system("PAUSE");
return 0; }
```

“Primeiro programa”

Questão 09. Determine a saída exata do programa a seguir e explique como o compilador C interpreta os argumentos do tipo caractere simples ('\\n', '\\t', '\\') passados para o modificador %c:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("%c%c%cPrimeiro programa", '\\n', '\\t', '\\'); /* O compilador interpreta o \\n como
    uma quebra de linha, o \\t como uma tabulação, e o \\ como uma aspa (para evitar
    problema de interpretação do sistema)*/

    printf("%c", "\\");
    system("PAUSE");
    return 0;
}
```

“Primeiro programa”

Questão 10. A Linguagem C é conhecida por ser sensível a caixa alta e baixa (case sensitive). Explique o significado prático desse conceito. Identificadores como 'peso', 'Peso' e 'PESO' representam a mesma variável na memória? Assinale a alternativa correta e complemente com sua justificativa:

a) Depende exclusivamente da implementação do compilador utilizado no sistema.

b) Verdadeiro (a linguagem C diferencia rigorosamente letras maiúsculas de minúsculas).

A linguagem C é case sensitive, isso significa que pra ela há uma distinção clara entre duas palavras que se apenas se diferenciam por ter uma letra maiúscula etc. exemplo: Para a linguagem c, uma comparação do tipo:

“Peso” == “peso” é false

“pEso” == “Peso” é false

“PESO” == “PESo” é false

A única forma dessa comparação ser verdadeira, é se elas forem 100% iguais, isto é, as letras maiúsculas e minúsculas estejam exatamente na mesma posição nas duas palavras, como no exemplo a seguir:

“PeSo” == “PeSo” é true

c) Falso (letras maiúsculas e minúsculas são interpretadas como equivalentes pelo compilador).

Questão 11. Para cada um dos valores constantes descritos na tabela abaixo, indique a classificação correta (por exemplo: constante inteira decimal, constante de ponto flutuante, constante de caractere, constante string ou sequência de escape) e o tipo de dado base correspondente em C (como char, int, float, double):

Constante Classificação (Tipo de Constante) Tipo Base em C

\r	[Sequencia de escape]	[]
2130	[Constante inteira decimal]	[int]
-123	[Constante inteira decimal]	[int]
33.28	[constante de ponto flutuante]	[float]
0XFA	[Constante inteira hexadecimal]	[int]
0101	[Constante binária]	[int]
2.0e30	[Constante inteira decimal]	[int]
\xDC	[Sequencia de escape]	[]
'\"'	[constante de caractere]	[char]
'\\'	[constante de caractere]	[char]
'F'	[constante de caractere]	[char]
0	[Constante inteira decimal]	[int]
'\0'	[constante de caractere]	[char]
"F"	[constante string]	[string]
-4567.89	[constante de ponto flutuante]	[float]

Questão 12. A declaração de variáveis define o tipo e o identificador de cada espaço reservado na memória. Analise cada uma das declarações na tabela a seguir, preencha o seu status (Correto ou Incorreto) e, caso seja incorreto, justifique detalhadamente o erro sintático:

a) int a; **Correto**

b) float b; **Correto**

c) double float c; **Incorreto.**

apenas um tipo pode ser atribuído a uma variável, ou ela é double ou ela é float, nunca pode ser os dois tipos ao mesmo tempo.

d) unsigned char d; **Correto**

e) unsigned e; **Correto** —> = unsigned int e;

f) long float f; **incorreto**.

long não pode ser utilizado no tipo float, apenas no tipo int e no tipo double

g) long g; **Correto** —> = long int e;

h) long double h; **Correto**

Questão 13. No desenvolvimento de programas em C, o que são conceitualmente os arquivos de inclusão (headers com extensão .h)?

a) São bibliotecas pré-compiladas em formato binário contendo funções estruturadas.

b) São utilitários do sistema que realizam a linkedição dos programas.

c) São arquivos de texto ASCII padrão contendo protótipos de funções, definições de constantes, macros e tipos.

d) São módulos de controle executados diretamente pelo microprocessador em tempo de execução.

Questão 14. Qual é o papel e o objetivo principal do programador ao incluir arquivos de cabeçalho (como <stdio.h>)?

a) Instruir o compilador a carregar as definições das funções da biblioteca padrão antes de compilar o código-fonte.

b) Linkeditar os arquivos binários do projeto automaticamente.

c) Executar e testar as saídas de vídeo diretamente no console de depuração.

d) Converter automaticamente o código-fonte C em arquivos executáveis (.EXE).

Questão 15. A diretiva #include, amplamente utilizada no topo dos arquivos C, é classificada como:

a) Uma instrução C nativa (compilada diretamente em linguagem de máquina).

b) Uma instrução específica de linguagens de programação orientadas a objetos.

c) Uma diretiva especial para o pré-processador C, executada antes da compilação.

d) Um objeto de classe de armazenamento dinâmico na memória heap.

Questão 16. As diretivas de pré-processador em C (todas iniciadas com o caractere #) são lidas e interpretadas pelo:

a) Linkeditor do sistema no momento de montagem do arquivo executável final.

b) Microprocessador diretamente em tempo de execução.

c) Pré-processador (fase do compilador que altera o programa-fonte antes da compilação propriamente dita).

d) Depurador integrado da IDE durante os testes de execução.

Questão 17. Dentre as instruções de escrita abaixo, quais estão sintaticamente corretas? O que essas variações demonstram sobre a flexibilidade de espaçamento e formatação do compilador C?

a) printf ("Primeiro programa");

b) printf("Primeiro programa");

c) printf("Primeiro programa");

d) printf "Primeiro programa" ;